Credit Card Customer Segmentation via Unsupervised Learning

Prepared by:
Lock Chun Hern
Lee Hong Jian

Date: 24 December 2025

Table of Contents

Section 1: Dataset Selection & Exploration

The dataset selected for this project is financial data, named "Credit Card Customer Data" from Kaggle shared by Habibimoghaddam (2023). This dataset provides valuable insights into credit card usage patterns and financial behaviour of credit card users. Each record represents an individual credit card holder, and there are 18 features for each record in the datasets.

1.1 Data Loading and Data Cleaning

The environment used for this project is Google Colab, with runtime type set as Python, CPU selected as hardware accelerator and run on the latest version. Necessary libraries and packages are first imported into the python script. The latest version of the data directory is imported from KaggleAPI, and the path to CSV is constructed using os library. The file is read using pandas read CSV function, with the first 5 rows and shape of the dataset displayed. The dataset contains 8950 rows and 18 features, as shown in Figure 1.1. Description of features is obtained from the Kaggle and presented in Figure 1.2.

```
# Check information of the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8950 entries, 0 to 8949
Data columns (total 18 columns):
 #   Column                            Non-Null Count  Dtype
---  ------                            --------------  -----
 0   CUST_ID                           8950 non-null   object
 1   BALANCE                           8950 non-null   float64
 2   BALANCE_FREQUENCY                 8950 non-null   float64
 3   PURCHASES                         8950 non-null   float64
 4   ONE_OFF_PURCHASES                 8950 non-null   float64
 5   INSTALLMENTS_PURCHASES            8950 non-null   float64
 6   CASH_ADVANCE                      8950 non-null   float64
 7   PURCHASES_FREQUENCY               8950 non-null   float64
 8   ONE_OFF_PURCHASES_FREQUENCY       8950 non-null   float64
 9   PURCHASES_INSTALLMENTS_FREQUENCY  8950 non-null   float64
 10  CASH_ADVANCE_FREQUENCY            8950 non-null   float64
 11  CASH_ADVANCE_TRX                  8950 non-null   int64
 12  PURCHASES_TRX                     8950 non-null   int64
 13  CREDIT_LIMIT                      8949 non-null   float64
 14  PAYMENTS                          8950 non-null   float64
 15  MINIMUM_PAYMENTS                  8637 non-null   float64
 16  PRC_FULL_PAYMENT                  8950 non-null   float64
 17  TENURE                            8950 non-null   int64
dtypes: float64(14), int64(3), object(1)
memory usage: 1.2+ MB
```

Figure 1.1: Information of Datasets

Id	Features	Description
1	**Cust_Id:**	Identification of credit card holder
2	**Balance:**	A credit card balance or Total amount left in their account to make purchases
3	**Balance_Frequency:**	How frequently the balance is updated, score between 0 and 1 (1 = frequently updated, 0 = not frequently updated)
4	**Purchases:**	Total amount of purchases made from account
5	**One_Off_Purchases:**	Maximum purchase amount done in one-go
6	**Installments_Purchases:**	Amount of purchase done in installment
7	**Cash_Advance:**	Cash in advance given by the user
8	**Purchases_Frequency:**	How frequently the Purchases are being made, score between 0 and 1 (1 = frequently purchased, 0 = not frequently purchased)
9	**One_Off_Purchases:** **Frequency:**	How frequently Purchases are happening in one-go (1 = frequently purchased, 0 = not frequently purchased)
10	**Purchases_Installments:** **Frequency:**	How frequently purchases in installments are being done (1 = frequently done, 0 = not frequently done)
11	**Cash_Advance_Frequency:**	How frequently the cash in advance being paid
12	**Cash_Advance_Trx:**	Number of Transactions made with "Cash in Advanced"
13	**Purchases_Trx:**	Number of purchase transactions made
14	**Credit_Limit:**	Limit of Credit Card for user
15	**Payments:**	Total amount of payments done by user
16	**Minimum_Payments:**	Minimum amount of payments made by user
17	**Prc_Full_Payment:**	Percentage of full payment paid by user
18	**Tenure:**	Tenure of credit card service for user

Figure 1.2 Description of Features

The DataFrame is checked for the presence of duplicated rows and null values. No duplicated rows are found for the DataFrame. 1 missing value is found in the “CREDIT_LIMIT” column and 313 missing values are found for “MINIMUM_PAYMENTS”. After checking the proportion, since the total missing value is less than 5% of total data, the records with missing value are dropped.

1.2 Descriptive Analysis

Descriptive statistics of each numerical column are printed for analysis. Distribution of each feature is plotted using seaborn as histogram as part of exploratory data analysis, shown in Figure 1.3. Correlation matrices are also plotted, as shown in Figure 1.4.

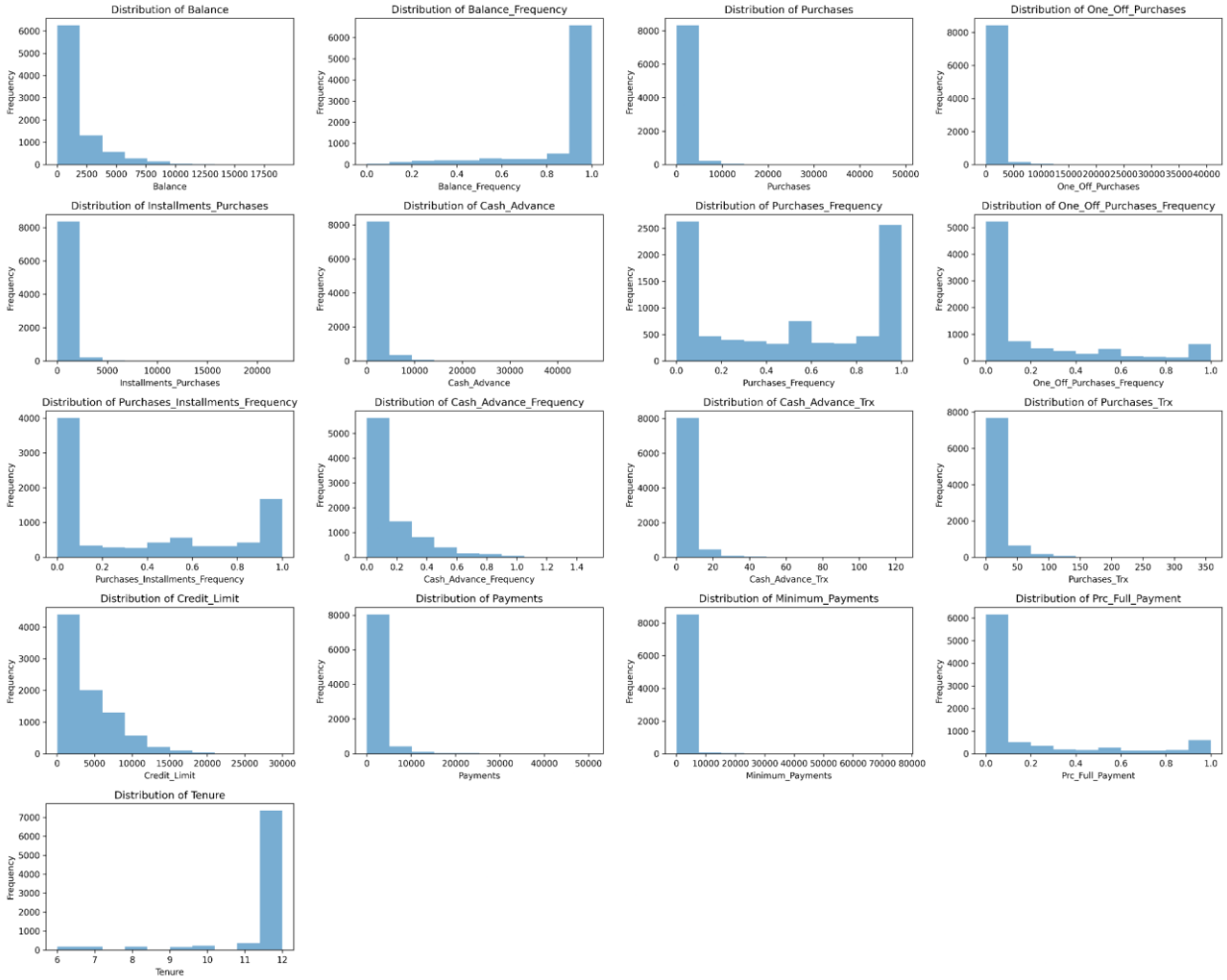

Figure 1.3: Distribution of Features

From the distribution of features, most of the features that are related to financial (e.g. Balance, Purchases, One_off_Purchase, Installments_Purchase, Cash_Advance, Payments, Minimum_Payments, Prc_Full_Payment) shows extreme right-skewness. This means most of the customers have relatively low values for these features, and only a small portion have exceptionally high values in these features. These outliers are likely to affect the clustering result, for example K-Mean clustering (distance-based) is very sensitive to outliers.

On the other hand, bimodal distribution is observed for Purchase_Frequency and. Purchase_Installments_Frequency. The number of customers for these features concentrated at two peaks, which can potentially be segmented into light users and heavy users of the credit cards based on their purchase pattern. Finally, distribution of balance frequency and tenure are left skewed, which means most of the customers update their balance frequently and have longer tenure. Standardization is also required for these skewed distributions.

1.3 Correlation Analysis

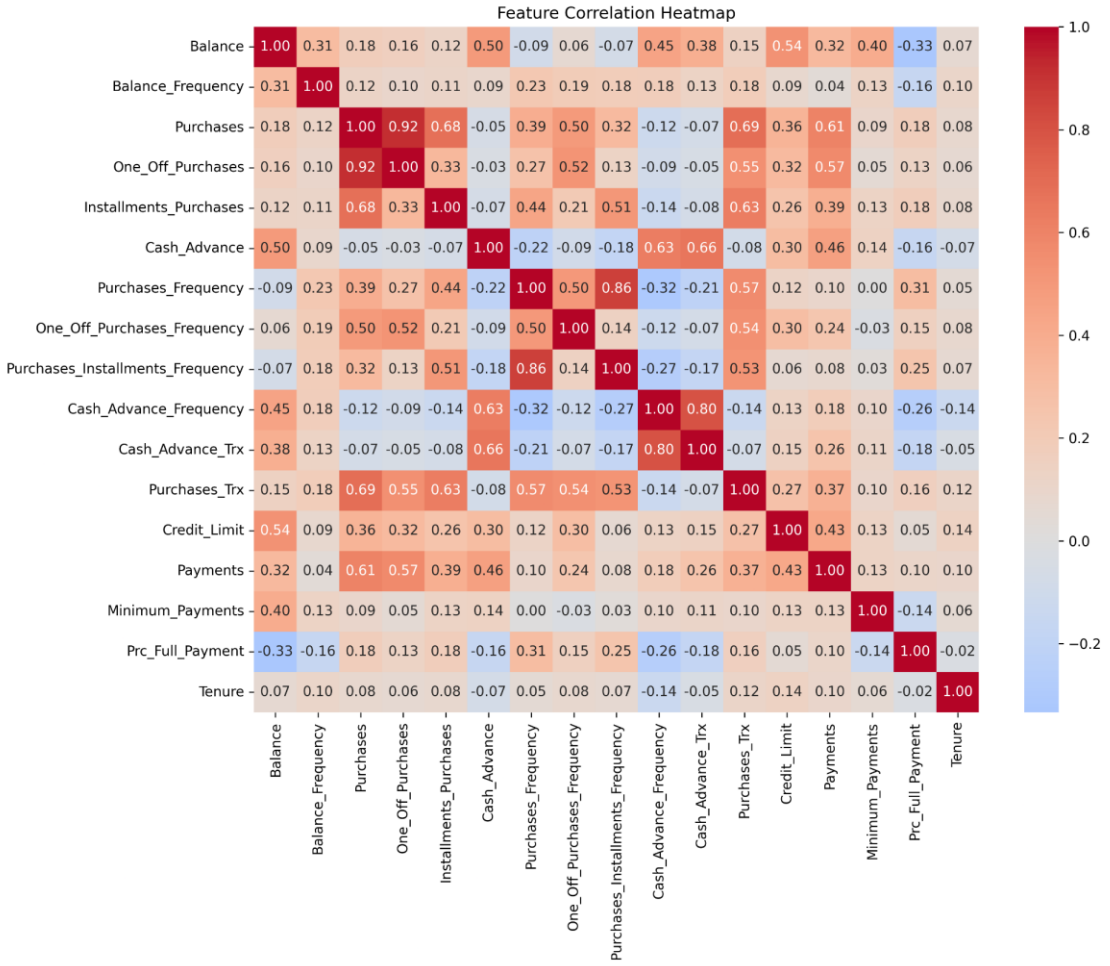

Figure 1.4: Correlation Matrices

In the correlation matrices, relationships between features are explored. High correlation is labelled as red while blue represents negative correlation. Low correlation is closer to white. From the diagram, it can be seen that Purchases and OneOff_Purchase are highly correlated (0.92). Purchases also have moderate positive correlation with Installments_Purchases (0.68) and Purchases_Trx (0.69). These correlations indicate multicollinearity, which may introduce redundancy and noise into the model, masking the other important features. This issue can be addressed by dimension reduction techniques like principal component analysis (PCA). PCA decorrelates the features and isolates primary components, which remove redundancy and reduce noise. Purchases have moderate positive correlation with Payments (0.61), together these may be used to segment the credit user with active spending behaviours.

On the other hand, there are some interesting relationships that are being revealed in the correlation matrices. The cash balance has moderate positive correlation (0.5) with cash_advance and credit_limit (0.54). This suggests that customers with higher balances likely have higher credit limits, and are more likely to withdraw cash in advance. Once again,

cash advance frequency and cash advance transaction also have positive correlation with balance, which follow closely behind Cash_Advance, indicating the need for dimension reduction to deal with these interrelated variables to prevent over-weighing these features. Balance also has a weak negative correlation (-0.33) with percentage of full payment. Tenure has a low correlation with the rest of features, which indicates it is not useful for segmentation.

1.4 Normalization

Normalization of features is done using RobustScaler imported from sklearn.preprocessing. Scaling is applied to make sure all features contribute fairly to the model. This method is chosen as the distribution of features is mostly skewed, and robust scaling is less affected by outliers. This is because robust scaler uses median and interquartile range (IQR) for scaling, unlike standard scaler which uses mean and standard deviation, and min max scaler which scales features based on minimum and maximum values.

Section 2: Baseline Clustering

This section will explore two different clustering methods: K-Means clustering and hierarchical agglomerative clustering for the financial dataset. Both methods are implemented with the optimal number of clusters (k) chosen, and a baseline comparison will be done to compare the results.

2.1 K-Means Clustering

K-Means algorithm is a distance based unsupervised learning algorithm where data points close to each other are grouped in a predefined number of clusters (k). K-Means clustering starts with random initialization of cluster centroids, then assigning each data point to the nearest centroid and recalculating the centroid based on assigned members of the clusters until convergence.

The optimal number of clusters (k) can be found out by using the elbow method. In this project, a range of 2 to 10 is used for the k values to find the optimal k. The steps of the elbow method are: First, iterating the K-Mean clustering algorithm over a range of k values. Secondly, for each k, compute inertia, which is the Within-Cluster Sum of Squares (WCSS). Third, plot the inertia against k values on a line chart. Lastly, choose the optimal k at the elbow. WCSS, referred to as inertia in scikit-learn, is the sum of squared distances between data points and their cluster centroids, which indicates how compact the cluster is. Low inertia means the data points are closer to their cluster centres. During K-means clustering, the value of k is optimized to balance cluster compactness and model complexity.

Silhouette score for each k is also computed and plotted on a line chart. Silhouette score is an internal evaluation metric measuring the quality of a cluster without true labels. It evaluates how compact a cluster is and how well-separated the clusters are. The score ranges from -1 to 1. The closer to 1 means the clusters are well-separated. 0 indicates there are overlapping clusters, and below 0 means the data points are closer to other clusters than their own.

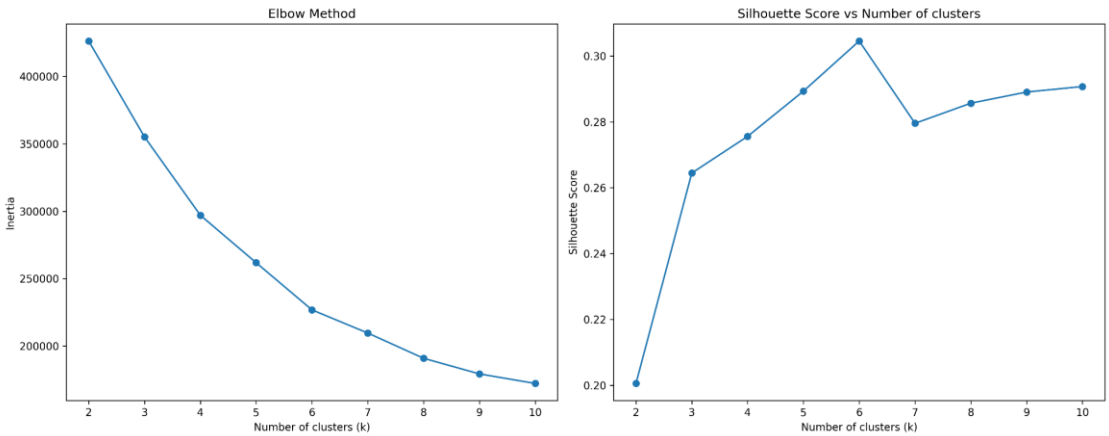

Figure 2.1: Linechart of Elbow Method and Silhouette Score

According to Figure 2.1 showing the linechart of the elbow method, there is no clear elbow as the inertia gradually decreases. This indicates the data does not have a well-separated cluster, likely due to high dimensionality of the data. Therefore, silhouette score is used to select the optimal k. The optimal number of clusters (k) is selected as 6 because it has the highest Silhouette score.

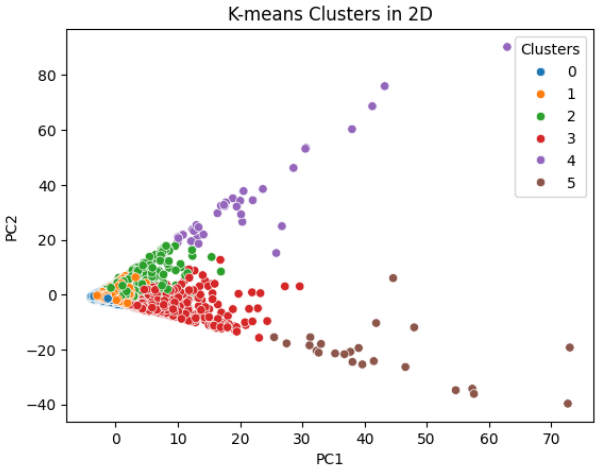

Figure 2.2: Visualization of K-Means clusters in 2D (k=6)

The clusters for k=6 are plotted in 2D using the first 2 principal components in PCA, showing varied cluster densities. From Figure 2.2, it can be seen that cluster 5 (brown) and cluster 4 (purple) have good separation from the main group but have low compactness. These clusters represent those extreme outliers in the dataset, which are proposed to be the high spender groups. In contrast, cluster 0 (blue) and cluster 1 (orange) are highly compact but overlap together, suggesting they share similar profiles of the first two principal components. Cluster 2 (green) and cluster 3(red) are more balanced in terms of compactness and separation from other clusters.

Cluster sizes and characteristics are further investigated by plotting a heatmap for comparison, as shown in Figure 2.3. Cluster 5 is a small cluster (23) containing the highest mean value in Purchases ($27,916), other related features (one off purchase, instalment purchase), credit limit and payments, which confirm the deduction that they are the high spending group. Cluster 4, on the other hand, is formed by 38 data points, which has high minimum payments ($27,540) but low purchases. However, the current mean balance for this group is moderate ($ 4,692). The high minimum payments are reflecting their payments for outstanding balance rather than current month's spending behaviour. This information indicates this group consists of at-risk or defaulted users, where it is likely that their debt has been accumulating for a long time and these debts are recently cleared when the bank has requested settlement of accumulated past amounts.

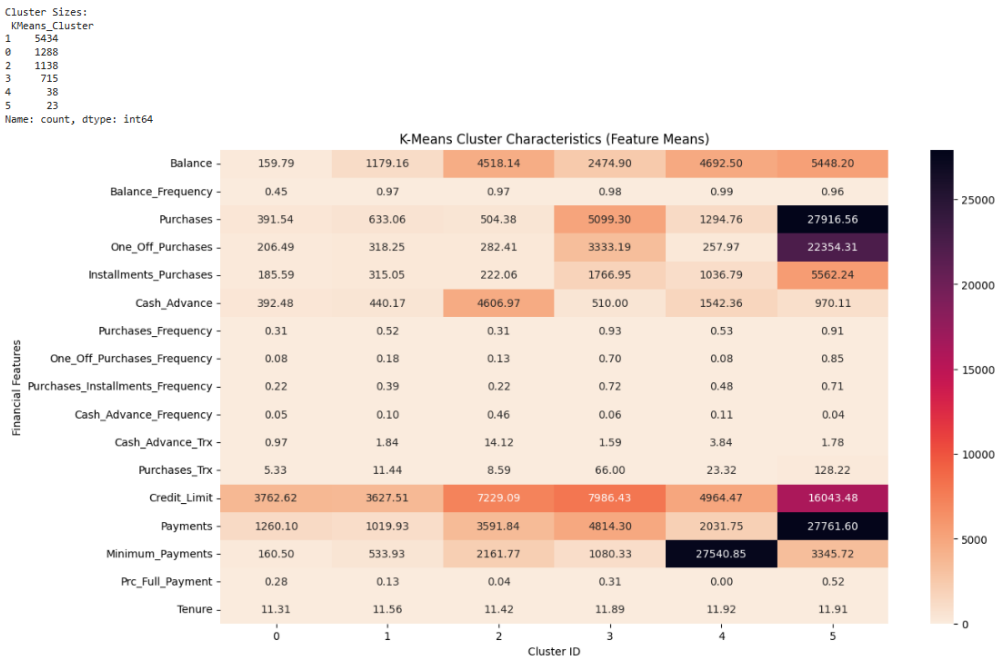

Figure 2.3: K-Means Cluster Sizes and Characteristics

Cluster 0 (1288 count) and cluster 1 (5434) have similar profiles in terms of credit limit and payments. These are likely the casual credit card users, consisting of around 80% of the datapoints. The significant difference between these clusters is the lower balance and purchases in cluster 0 compared to cluster 1. Cluster 2 (1138 users) are formed by credit card users with high cash advance ($4,606) behaviour. They also have moderately higher balance, credit limits and payment than average users. These groups are users that are heavily reliant on cash rather than making purchases directly with a credit card, utilising cash advance for liquidity. Cluster 3 (715 users) has higher mean purchase ($5099), credit limit and payment than average users. These are the active customers that are contributing most to the bank's transactional volume and revenues. Cluster 4 is formed special group of customers with high minimum payments ($27540) but low purchase($1294). Cluster 5 are the high spender with high purchase ($27916) and high payments.

In summary, the clusters characteristic reveals different groups of spenders. Cluster 0 are formed by low spenders or non-active users, cluster 1 are casual users, cluster 2 are cash borrowers, cluster 3 are high value customers, cluster 4 is at risk users and cluster 5 is extremely high spenders. This will help the bank to manage the risk and assign marketing and promotion to the right target group. However, the cluster quality can be improved as there are still overlapping components, and using all the features contribute noise to the clustering algorithm.

2.2 Hierarchical Agglomerative Clustering

Hierarchical Agglomerative Clustering is an algorithm where clusters are iteratively combined in a hierarchical manner using a bottom up approach. This algorithm starts by assigning each data point as one cluster, then iteratively merges two clusters that are closest to each other using distance metrics defined by the chosen linkage method. This process continues until all points are merged into a single cluster. The merging sequence can be visualized in a dendrogram, allowing identification of optimal number of clusters, through observation of vertical gaps representing separation or dissimilarity between merges.

In this project, 3 different linkage methods (ward, complete, average) of this clustering method are explored. The agglomerative clustering algorithm and visualization tools are imported from sklearn.cluster and scipy libraries. The dendrogram of each method is shown in Figure 2.3.

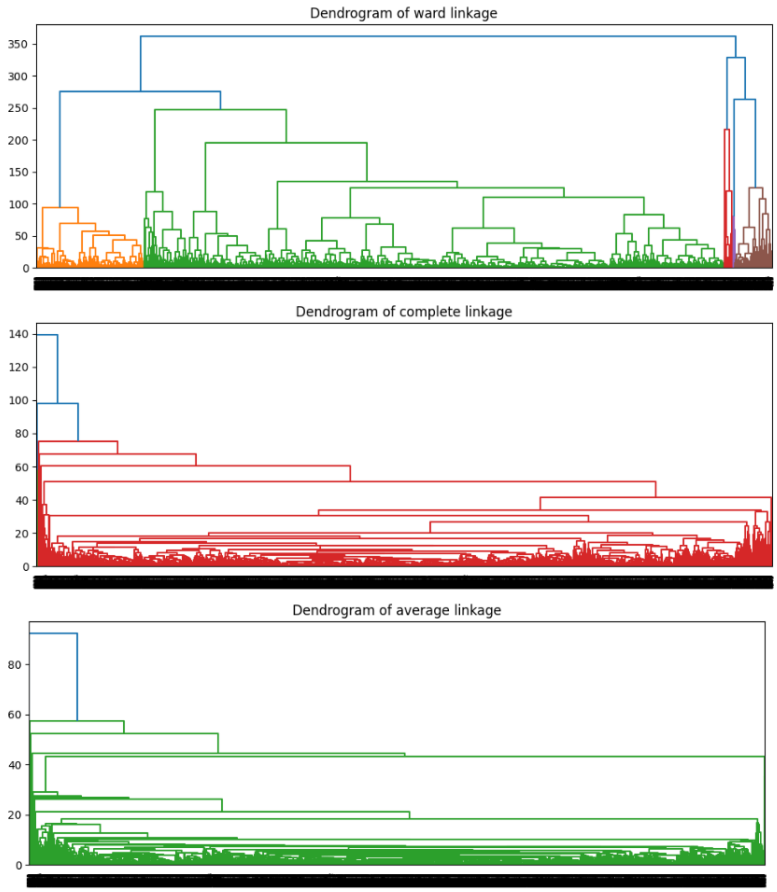

Figure 2.4: Dendrograms of Hierarchical Agglomerative Clustering

Analysis of the dendrograms in Figure 2.4 reveal that ward linkage produces the most balanced cluster hierarchy, with clear branching at higher distances. In contrast, complete linkage and average linkage produce a single massive cluster (indicated by red line and green line respectively) with a second cluster containing some extreme outliers. This is confirmed by checking the percentage distribution of clusters in Figure 2.5. The results show that by

using complete and average linkage methods with k = 2,3,4, more than 99.6% of data points are clumped in cluster 0, while other clusters only contain some outliers.

Cluster		0	1	2	3
Method	K				
average	2	99.953682	0.046318	-	-
	3	99.942103	0.046318	0.011579	-
	4	99.687355	0.254748	0.011579	0.046318
complete	2	99.918944	0.081056	-	-
	3	99.698935	0.081056	0.220009	-
	4	99.687355	0.081056	0.220009	0.011579
ward	2	6.519222	93.480778	-	-
	3	93.480778	5.257063	1.262158	-
	4	5.257063	78.809634	1.262158	14.671144

Figure 2.5 Percentage Distribution of Clusters

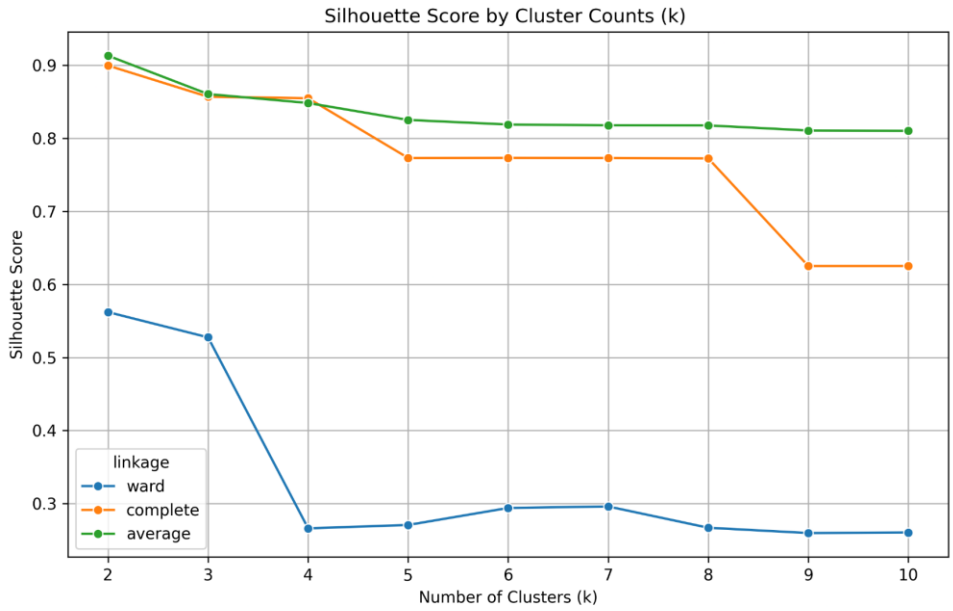

Figure 2.6: Silhouette score by Cluster Counts using different Linkage Methods

Figure 2.6 shows the Silhouette score of different cluster counts across different linkage methods. At first glance, average linkage methods appear to be superior as they have high silhouette scores of >0.8, peaking at around 0.9 for k= 2 and 3. However, the high silhouette score likely resulted from separation (distance) between the extreme outliers and the main cluster as stated in the analysis of dendrogram and cluster distribution. This masks the internal variance of the majority population, failing to identify meaningful segments within the customers. The analysis result is similar for the complete linkage methods.

On the other hand, the ward linkage method produces a moderate silhouette score of > 0.5 at k=2 and k=3. While the score is lower than other methods, ward linkage prioritises the minimization of within cluster variance, which leads to a more balanced cluster (e.g. at k= 2,

6.5% for cluster 0 and 93.5% for cluster 1). The Silhouette score drops sharply to a low score ranging between 0.25 to 0.30 when the k is increased to 4 and above, indicating that the clusters formed might have more overlapping features. While further segmentation may be practical in business context to obtain more granular customer segments, the low Silhouette score signals low quality (overlapping) clusters, which might be questionable for decision making by the bank stakeholders.

Therefore, ward linkage is the better choice of linkage method for the dataset. The optimal height to cut the dendrogram is at height 350 as the largest vertical gap in the ward dendrogram, resulting in the number of clusters (k) selected as 2. This decision is also justified by the fact that k=2 is the highest Silhouette score in ward linkage method. The cluster characteristics for this configuration (method= 'ward', k=2) are further analysed as shown in Figure 2.7.

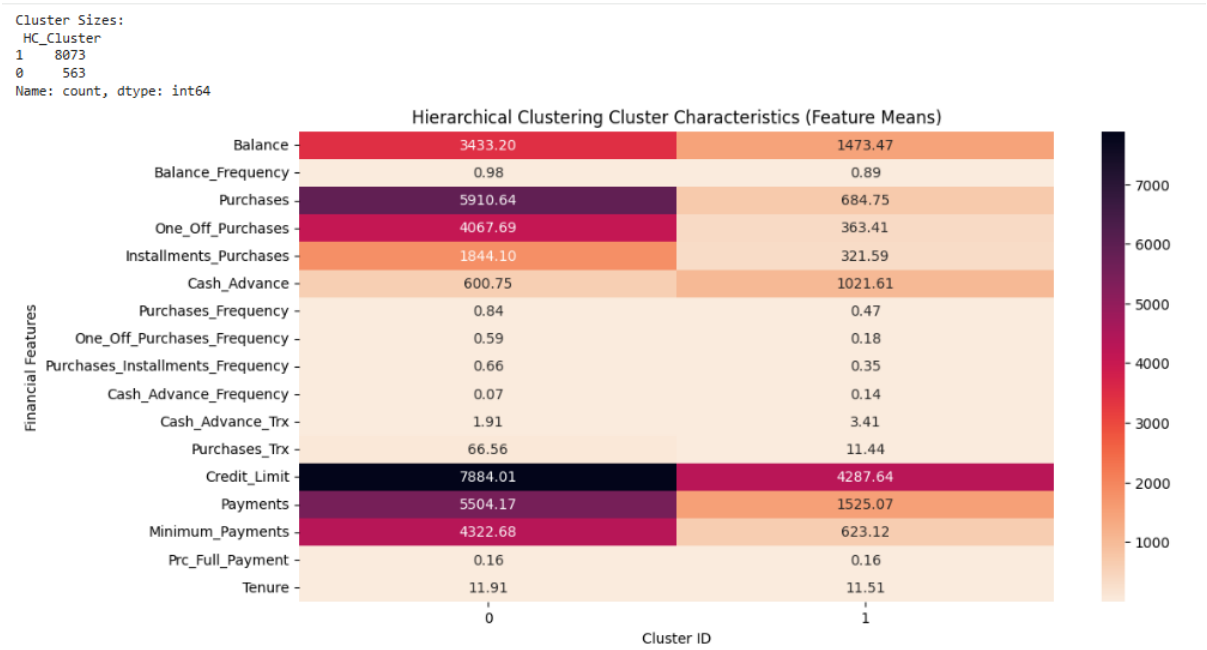

Figure 2.7 Cluster Characteristics of Hierarchical Clustering with Ward Linkage

The clusters characteristics describe two most distinct customer groups: high spenders and casual users. Cluster 0 has 563 users (6.5%), with higher mean balance ($ 3433), purchase ($5910), purchase transaction (66), credit limit ($7884) and payments ($5504). Cluster 1 consists of 8073 users (93.5%), which contains characteristics of a casual credit card having lower mean balance of $1473, purchases ($ 684), credit limit ($4287) and payments ($1525). It is notable that cluster 1 has a higher cash advance ($ 1021) and frequency, indicating that high spender users are more likely to make purchases with direct credit cards and rely less on getting cash for liquidity.

2.3 Comparison between Results of K-Means and Hierarchical Clustering

The comparison between results of K-Means and Hierarchical Agglomerative Clustering presented in Table 2.1, with three different evaluation metrics.

Table 2.1 Results Comparison Table

Methods	**Optimal k**	**Silhouette**	**Davies-Bouldin**	**Calinski-Harabasz**
K-Means	6	0.304564	1.116030	1854.272572
Hierarchical Clustering (linkage= “ward”)	2	0.562084	1.58727	1398.693025

As discussed in the subsection above, silhouette score evaluates two aspects for the clusters: The mean intra-cluster distance (a), calculated by the average distance between each point in the same cluster; The mean nearest-cluster distance (b), calculated by the average distance from a data point to the points in the nearest neighbouring clusters. The Silhouette Coefficient for a sample is (b-a) / max(a,b) (Scikit-learn developers, 2025). The score ranges from -1 to 1. A score close to 1 means the clusters are dense and well-separated from each other.

Davies-Bouldin score (DBI) is defined as the average similarity measure of each cluster with its most similar cluster, where the similarity is the ratio of within-cluster distances to between cluster distances. The within-cluster distance is calculated by the average distance between each point to the cluster’s centroid (average position of points), while the between-cluster distance is calculated by distance between centroids of every pair of clusters. The minimum score is zero and lower score indicates better clustering results.

Calinski-Harabasz Index (CHI) is the ratio of sum of between-clusters dispersion and within-cluster dispersion for all clusters. The cohesion is estimated based on distances from the datapoints in a cluster to its cluster centroids, while separation is based on the distance of cluster centroids from the global centroid. Similar to Silhouette score, higher score means clusters are dense and well separated.

From the table, it can be seen that from the perspective of Silhouette score, hierarchical clustering at k=2 has better results than K-Means at k=6, scoring higher at 0.56. However, K-Means clustering scores are better in Davies-Bouldin score (1.11) and Calinski-Harabasz Index (1854). These conflicting results suggest that while hierarchical clustering has better separation between clusters, it might have higher internal variance, which means the clusters are less compact compared to clusters in K-Means clustering. This is indicated by the results of the two metrics (DBI and CHI) that rely on centroid-based distance. These distinctive

results are largely driven by the choice of optimal k rather than the difference in the model itself, where 2 clusters in the dataset provide a cleaner split while 6 clusters produce a more cohesive group.

To choose a better performing model in this case largely depends on business context and objectives. If the aim of the business questions require granularity for targeted marketing, K-Means at k=6 provide a better segmentation of the customer base. On the other hand, if the primary aim is to differentiate the high spenders from casual users, hierarchical clustering at k=2 would be a better performing model.

Section 3: Dimensionality Reduction

Dimensionality reduction techniques reduce the number of features/attributes while still keeping a high percentage of information in the original information. Dimensionality reduction simplifies the complexity of the original dataset and improves the performance of clustering algorithms without losing the ability to interpret underlying patterns. In this section, PCA and UMAP will be used to perform the dimensionality reduction for the dataset. PCA is most effective for linear relationships between the variables whereas UMAP works better for complex and non-linear relationships in the dataset. This section will outline the process of implementation of these two techniques to transform the high-dimensional credit card dataset to a lower dimensional representation, eliminating noise and multicollinearity.

3.1 Principal Component Analysis (PCA)

Principal Component Analysis is a dimensionality reduction technique that performs feature transformation to reduce the number of features in the data set while retaining as much information as possible. Explained variance ratio for PCA represents how much of variance (information) is captured by each of the principal components. PCA helps to reduce the complexity of the dataset by focusing on the most informative variables. This helps to improve the model performance and reduce overfitting issues. The original dataset contains 17 features that lead to data sparsity and distance convergence. PCA transforms correlated variables into the set of linearly uncorrelated principal components. It filters out the noise and preserves the latent behavioural signal of the credit card users. The explained variance ratio is used to determine the optimal number of dimensions that are required to describe the current dataset effectively.

```
# Principal Component Analysis (PCA) ---

# Apply PCA to find variance thresholds
pca_full = PCA().fit(X) # X is the scaled data from Part 2
cum_var = np.cumsum(pca_full.explained_variance_ratio_)

# Determine components for 90%, 95%, and 99% variance
comp_90 = np.argmax(cum_var >= 0.90) + 1
comp_95 = np.argmax(cum_var >= 0.95) + 1
comp_99 = np.argmax(cum_var >= 0.99) + 1

print(f"Components for 90% Variance: {comp_90}")
print(f"Components for 95% Variance: {comp_95}")
print(f"Components for 99% Variance: {comp_99}")
```

```
Components for 90% Variance: 7
Components for 95% Variance: 9
Components for 99% Variance: 13
```

Figure 3.1: Initialization of Principal Component Analysis

Figure 3.1 illustrates that the components for 90% variance are 7 components, components for 95% variance are 9 components and components for 99% variance are 13 components. Figure 3.2, 3.3 and 3.4 shows PCA Explained Variance Ratio with different thresholds.

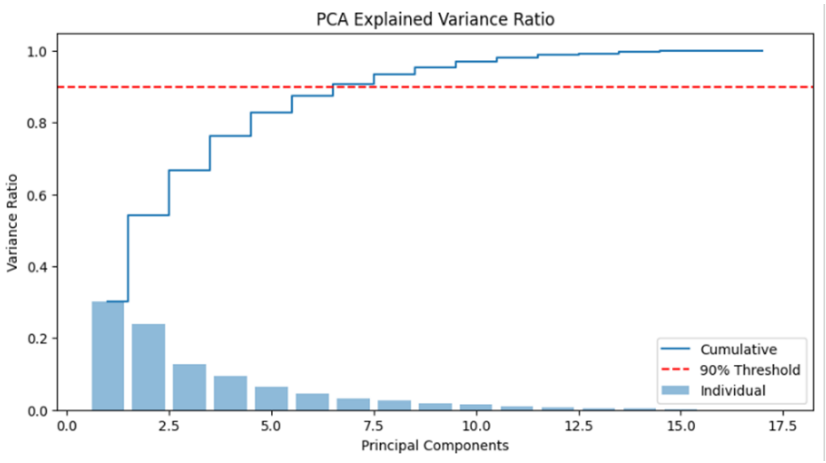

Figure 3.2 PCA Explained Variance Ratio (90% variance)

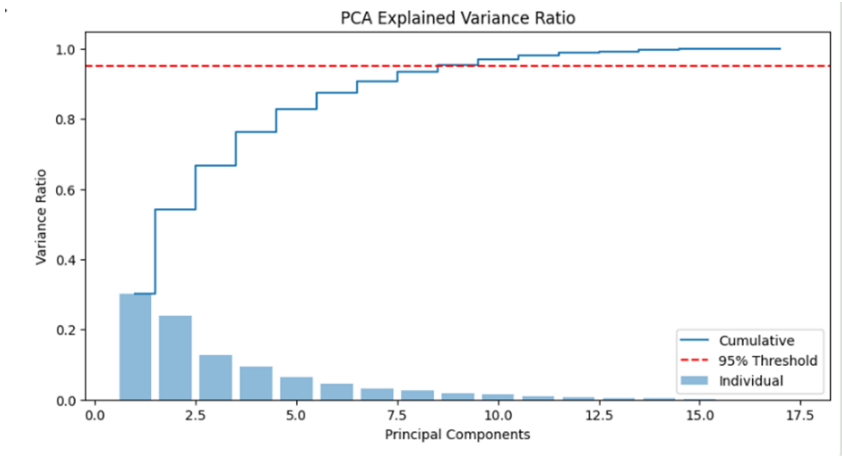

Figure 3.3 PCA Explained Variance Ratio (95% variance)

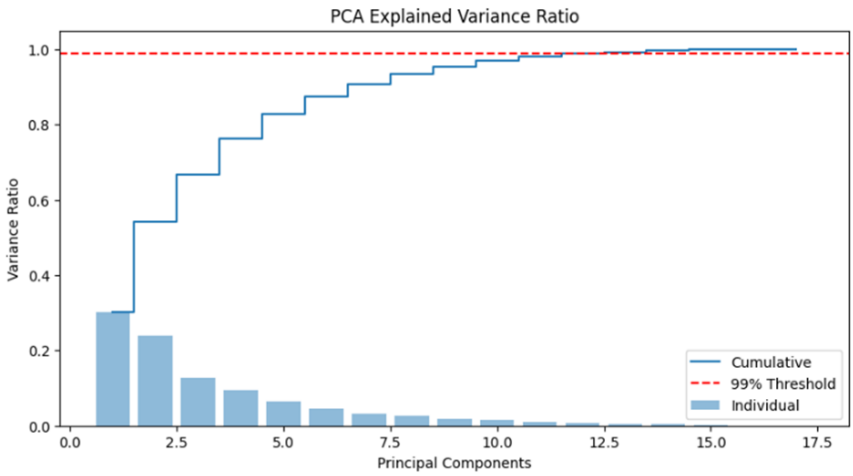

Figure 3.4 PCA Explained Variance Ratio (99% variance)

The 90% threshold results indicate significant redundancy from the original dataset, as most of the variance is retained by using only 7 components from the dataset. At the 90% threshold, PCA discards nearly 60% of the total features while losing only 10% information.

This denoising process is important to stabilizing the subsequent clustering centroids. The 95% threshold results provide a balanced middle ground that has reduced the feature from 17 17 to 9 while still keeping 95% of the information. In contrast, the 99% threshold is overly conservative, as almost all the information is kept but the dimensions are not significantly reduced (17 to 13).

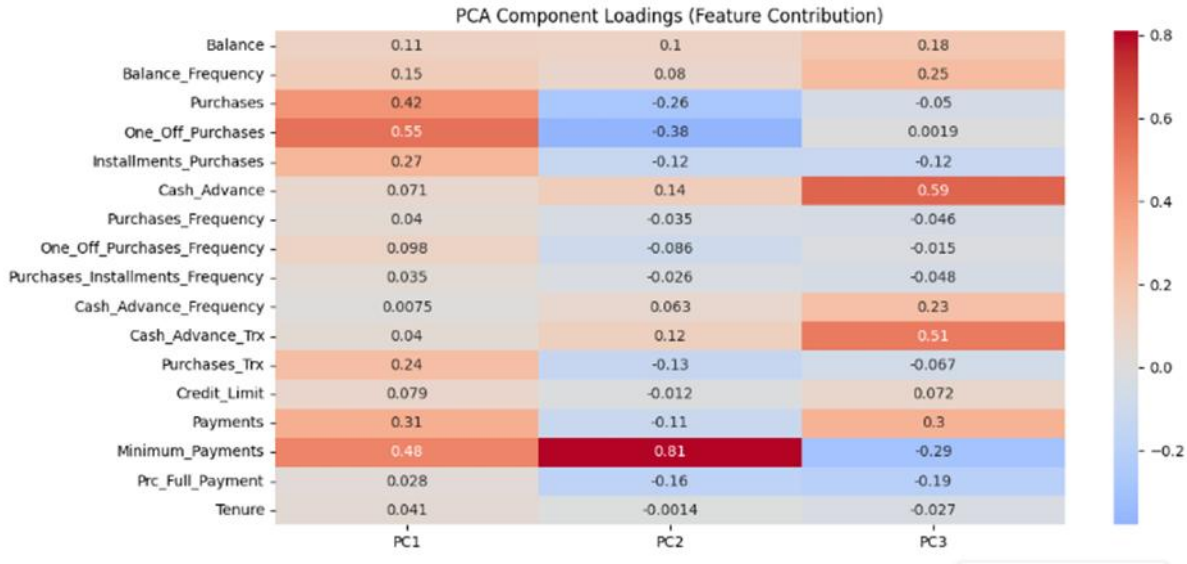

Figure 3.5: PCA Component Loading

```
--- Top 5 Contributing Features for each PC ---

Top Features for PC1:
One_Off_Purchases        0.548575
Minimum_Payments         0.483155
Purchases                0.417819
Payments                 0.314027
Installments_Purchases   0.272741
Name: PC1, dtype: float64

Top Features for PC2:
Minimum_Payments     0.808844
One_Off_Purchases    0.378730
Purchases            0.258500
Prc_Full_Payment     0.163263
Cash_Advance         0.139348
Name: PC2, dtype: float64

Top Features for PC3:
Cash_Advance         0.589853
Cash_Advance_Trx     0.512878
Payments             0.298098
Minimum_Payments     0.293205
Balance_Frequency    0.245521
Name: PC3, dtype: float64
```

Figure 3.6 Top 5 Contributing Features for each Principal Components

```
print("\n--- Top Contributing Features for each Principal Component ---")
for pc in ['PC1', 'PC2', 'PC3']:
    # Get the feature with the highest absolute loading value
    top_feature = loadings[pc].abs().idxmax()
    loading_val = loadings.loc[top_feature, pc]
    print(f"{pc}: Most influenced by '{top_feature}' (Loading: {loading_val:.4f})")
```

```
--- Top Contributing Features for each Principal Component ---
PC1: Most influenced by 'One_Off_Purchases' (Loading: 0.5486)
PC2: Most influenced by 'Minimum_Payments' (Loading: 0.8088)
PC3: Most influenced by 'Cash_Advance' (Loading: 0.5899)
```

Figure 3.7 Top Contributing Feature for each Principal Component

Figure 3.5 presents the heatmap of the PCA component loading, which shows the feature contributions (loadings) for each principal component. This illustration reveals how the PCA reduces the dimensions and which features contribute the most to each principal component. By analysing the top 5 contributing features for each principal component, the fundamental characteristics of the particular principal component can be defined. The top contributing features can be referred at Figure 3.6 and Figure 3.7.

PC1 represents the purchase dimension, which is strongly affected by the One_Off_Purchases (0.55) and Purchases (0.42). This component contains information about the active purchasing behaviour of the customer. PC2 is heavily determined by the Minimum_Payment feature (0.81). This component isolates the customer repayment pattern and debt-related behaviour, which can be used to distinguish those who make full payments and those who only pay a minimum amount. From a risk management perspective, this component can reveal an at-risk customer segment that only pays a minimum amount and at higher risk of defaulting. PC3 represents the dimension of cash liquidity, which is highly affected by the Cash_Advance (0.59) and Cash_Advance_Trx (0.51). This component can be used to identify specific subgroups of the customers who are using their credit card to withdraw cash instead of making direct payment through card.

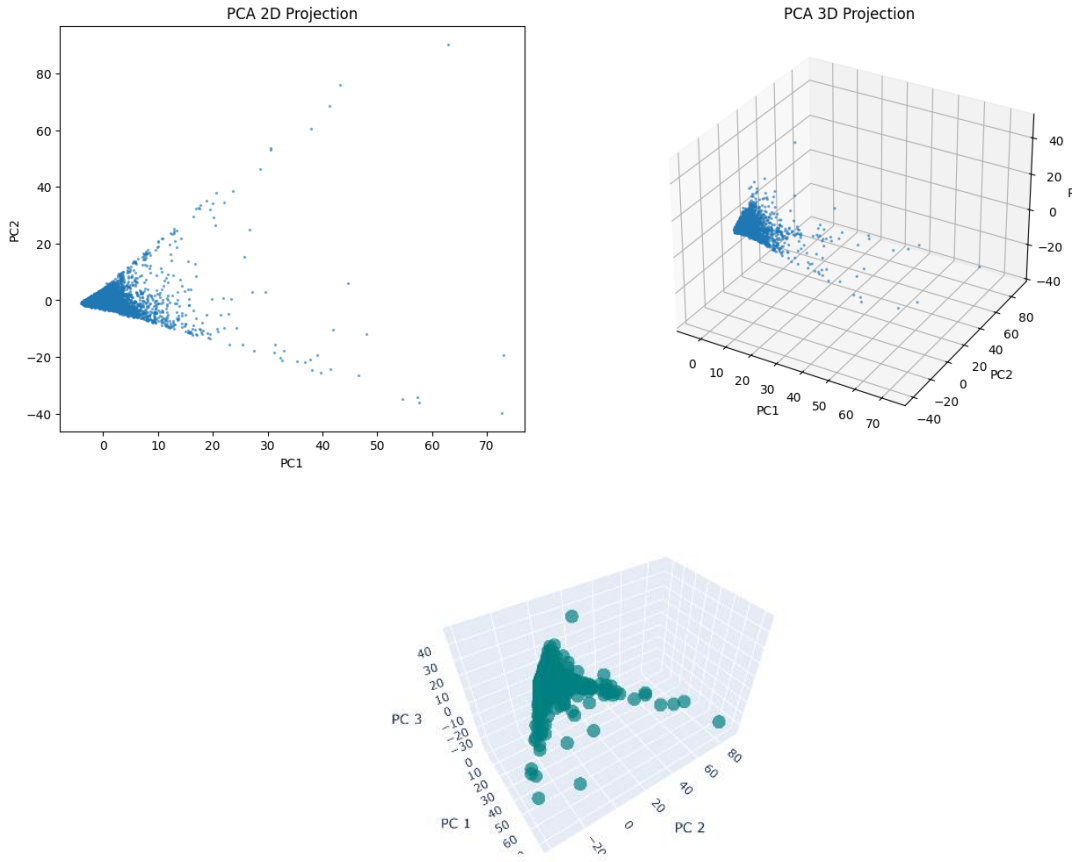

Figure 3.8 Visualization of data in 2D and 3D using Top Principal Components

In this section, 2D and 3D visualization of the dataset using the top principal components to explore the reduced dataset. The PCA 2D projection shows that the data plotted against PC1 and PC2 on x-axis and y-axis respectively. A dense core of the data points at the origin (0,0) is revealed, likely representing the average credit card users. From the core, two distinct ‘arms’ stretch outward, forming a characteristic V-shape. These arms were formed from datapoints that represent the different customer behaviours: one is directed toward principal component 1 (high spenders) and another toward principal component 2 (minimum payers). The 3D PCA projection added the third dimension which represents cash liquidity (cash advance), which allows us to review the overlapping 2D projections, separated by their cash withdrawal habits. The V-shape structure observed in the 2D and 3D projection is very common in the financial data where a portion of the customers is more extreme on specific behaviours such as exclusive high spenders, minimum payers and cash-advance users, where they tend to follow the specific path away from the common behaviour of average customers.

3.2 UMAP

UMAP (Uniform Manifold Approximation and Projection) is the optimization process for the non-linear dimensional reduction technique based on algebraic topology and Riemannian geometry. Unlike PCA which assumes linear relationships, UMAP assumes that the data lies on a manifold (curved shape) and optimizes a low-dimensional representation to preserve the shape. UMAP focuses on local connectivity between data points, and works well to capture complex, non-linear structures found in financial data.

In this section, UMAP is applied with different parameter settings of n_neighbors (5,10,50,100) and different min_dist values (0.1,0.5,0.9) as shown in Figure 3.9. The number of nearest neighbors is the local connectivity that is controlled by the n-neighbours parameter. Lower values prioritize local connectivity, resulting in more fragmented clusters. Minimum distance determines how tightly UMAP packs points together. Lower value of min_dist results in more compact clumps while higher value preserves the shape of the distribution, like the V-shape mentioned above.

Visualization of the UMAP outcome is done, as shown in Figure 3.10. To ensure the reproducibility of the non-linear projections, a fixed random state is allocated for the UMAP optimization process. This limits the computation to a single thread, but guarantees the consistency of the spatial orientation of the clusters across the iteration.

```
# UMAP Hyperparameter Exploration (Grid Search) ---
neighbors = [5, 15, 50, 100]
dists = [0.1, 0.5, 0.9]
best_umap_score = -1
X_umap_best = None
best_params = {}

fig, axes = plt.subplots(4, 3, figsize=(18, 22))
for i, n in enumerate(neighbors):
    for j, d in enumerate(dists):
        reducer = umap.UMAP(n_neighbors=n, min_dist=d, n_components=2, random_state=42)
        u_emb = reducer.fit_transform(X)

        # Test Silhouette for this projection to select best one for Part 4
        # We find best K for this specific projection
        k_temp = find_optimal_k(u_emb)
        labels = KMeans(n_clusters=k_temp, random_state=20).fit_predict(u_emb)
        score = silhouette_score(u_emb, labels)

        axes[i, j].scatter(u_emb[:, 0], u_emb[:, 1], s=1, c=labels, cmap='viridis')
        axes[i, j].set_title(f"n={n}, d={d}, k={k_temp}")
        axes[i, j].axis('off')

        if score > best_umap_score:
            best_umap_score = score
            X_umap_best = u_emb
            best_params = {'n': n, 'd': d, 'k': k_temp}

plt.tight_layout()
plt.show()
print(f"Best UMAP Params: {best_params}")
```

Figure 3.9 Dimension Reduction with UMAP

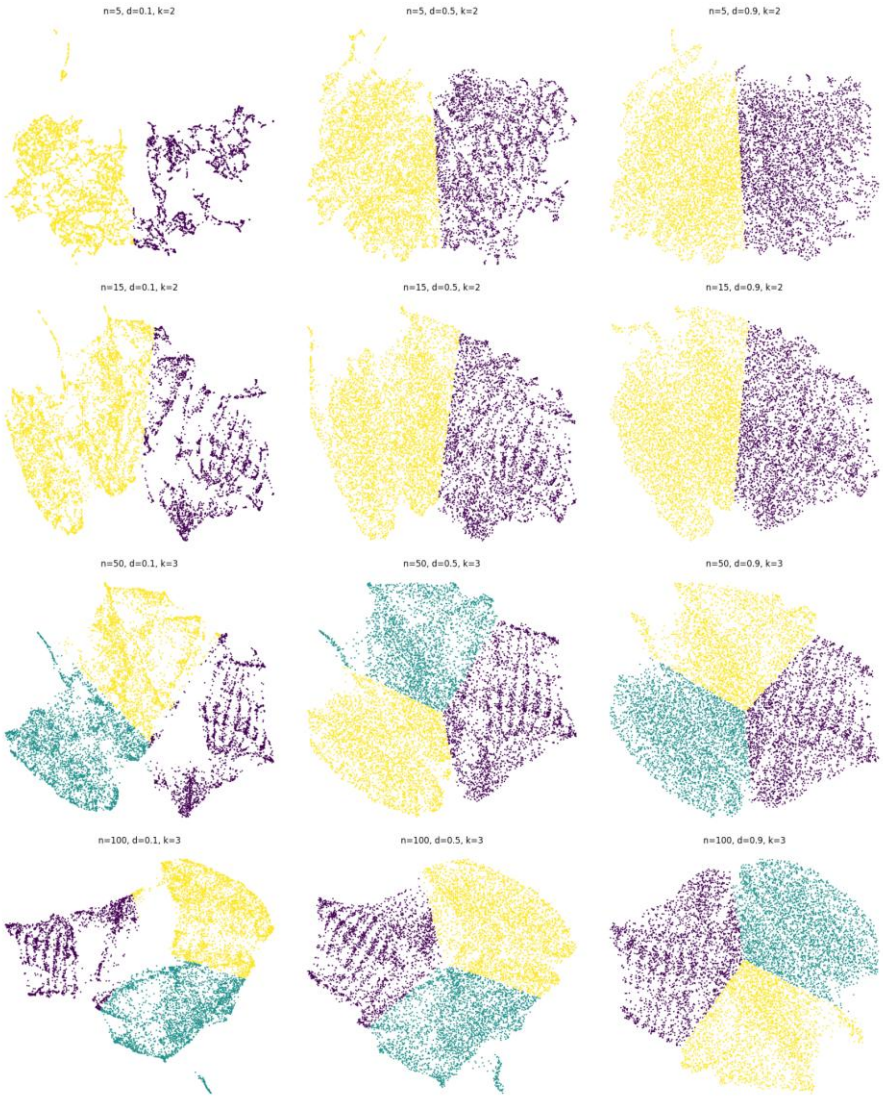

Figure 3.10 Visualization of UMAP with different parameters

From Figure 3.10, the first column, it can be seen that when n_neighbors =5, the data are more fragmented, where the strong local connectivity creates string-like structure and disconnected pockets. This parameter is best to keep at low value if the objective is to get very specific, micro segments of customers. As the number of neighbours (n) increases (from top to bottom), the data merges into a more cohesive structure, providing a more stable foundation clustering algorithm to find customer segmentation.

At d=0.1, the clusters are dense, compact and clumped together, which results in clear separation. As the minimum distance (d) increases (to the right), the gap between the data points widens, and the gaps between different coloured regions reduce, blurring the boundaries between groups. This makes clustering algorithms like K-Means more difficult to find distinct clusters.

Additionally, grid search is applied in the UMAP to find the best hyperparameters determined by the same performance metric in part 2 (Silhouette score). The reason for using grid search is to find the best combination for the parameter for UMAP as it provides few options for neighbors (5, 15, 50, 100) and distance (0.1, 0.5, 0.9). By using grid search (test for all the combinations), we will be able to know which of the combinations is the best parameter evaluated by the performance metric (Silhouette score). K-Means clustering is used for the process to determine optimal k because it is faster.

```
--- UMAP PARAMETER SELECTION TABLE ---
```

	n_neighbors	min_dist	Optimal K	Silhouette	Davies-Bouldin	Calinski-Harabasz
0	5	0.1	2	0.4832	0.8229	10325.956055
9	100	0.1	3	0.4794	0.7284	11026.430664
3	15	0.1	2	0.4750	0.8048	10653.478516
6	50	0.1	3	0.4498	0.8029	9848.436523
1	5	0.5	2	0.4320	0.9160	8630.246094
4	15	0.5	2	0.4203	0.9280	8319.696289
10	100	0.5	3	0.4170	0.8058	7999.187988
2	5	0.9	2	0.4043	0.9781	7456.424805
7	50	0.5	3	0.4022	0.8697	7759.020996
5	15	0.9	2	0.3960	0.9965	7154.366211
11	100	0.9	3	0.3912	0.8806	7317.455078
8	50	0.9	3	0.3849	0.8902	7073.314453

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning:

n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
```

Figure 3.11 UMAP Parameter Selection from GridSearch

Based on the UMAP parameter selection table, n_neighbors=5 and min_dist=0.1 is the best combination, as it has better performance in the Silhouette, Davies-Bouldin and Calinski-Harabasz compared to other combinations.

Section 4: Enhanced Clustering/ Impact to Clustering

In this section, the dataset with reduced dimension by PCA and UMAP will be applied to K-Means clustering and Hierarchical clustering to choose the best model. The impact to the clustering result by the dimension reduction techniques will be investigated.

4.1 Implementation of K-Means and Hierarchical Clustering on Reduced Features

First, a function is defined as shown in Figure 4.1, to achieve several goals. This function finds the optimal k for the specific method (K-Means or Hierarchical clustering), then returns the same evaluation metrics as Part 2. This function incorporates dynamic k selection for k values between 2 to 10, because the dimension reduction techniques may cause optimal k to shift from the k value used in Section 2. The k is selected based on Silhouette score.

```
def evaluate_all_metrics(data, method, dim_name):
    """Finds optimal k for the data based on the specific method and returns metrics."""
    best_k = 2
    best_sil = -1

    # 1. Dynamic K-Selection (2-10) - Method Specific
    for k in range(2, 11):
        if method == "K-means":
            test_model = KMeans(n_clusters=k, random_state=20)
        else:
            # For Hierarchical, we use AgglomerativeClustering to find the best k
            test_model = AgglomerativeClustering(n_clusters=k)

        lbls = test_model.fit_predict(data)
        score = silhouette_score(data, lbls)

        if score > best_sil:
            best_sil = score
            best_k = k

    # 2. Final Execution with the Optimal k found for THIS specific method
    start_t = time.time()
    if method == "K-means":
        final_model = KMeans(n_clusters=best_k, random_state=20)
    else:
        final_model = AgglomerativeClustering(n_clusters=best_k)

    final_labels = final_model.fit_predict(data)
    runtime = time.time() - start_t

    return {
        "Method": method,
        "Dim Reduction": dim_name,
        "Silhouette": round(silhouette_score(data, final_labels), 4),
        "Davies-Bouldin": round(davies_bouldin_score(data, final_labels), 4),
        "Calinski-Harabasz": round(calinski_harabasz_score(data, final_labels), 4),
        "Optimal k": best_k,
        "Runtime": round(runtime, 4)
    }, final_labels
```

Figure 4.1 Function to Determine Optimal k for Specific Method and Returns Metrics

```
# Ensuring we use the thresholds and best UMAP params calculated in Part 3
pca_90 = PCA(n_components=comp_90).fit_transform(X)
pca_95 = PCA(n_components=comp_95).fit_transform(X)
pca_99 = PCA(n_components=comp_99).fit_transform(X)

# Best UMAP (using X_umap_best from Part 3 Selection Table, assume same param for both models)
scenarios = [
    (X, "None"),
    (pca_90, "PCA (90%)"),
    (pca_95, "PCA (95%)"),
    (pca_99, "PCA (99%)"),
    (X_umap_best, "UMAP")
]

comparison_results = []
best_overall_labels = None
```

```
# Run Experiments to find the best results
for data_input, label in scenarios:
    # Run K-means variant
    metrics_km, labels_km = evaluate_all_metrics(data_input, "K-means", label)
    comparison_results.append(metrics_km)

    # Run Hierarchical variant
    metrics_hc, labels_hc = evaluate_all_metrics(data_input, "Hierarchical", label)
    comparison_results.append(metrics_hc)
```

Figure 4.2 Definition of Parameters for Dimension Reduction

Next, dimension reduction techniques are defined illustrated by Figure 4.2. Baselines are performed with the same specification in Section 2.0. 3 different thresholds (90%, 95%, 99%) are used for PCA. The parameters for UMAP are set based on the best parameters from GridSearch results in Section 3.0. Dimension reduction is applied to the normalized X to reduce the features.

```
# Run Experiments to find the best results
for data_input, label in scenarios:
    # Run K-means variant
    metrics_km, labels_km = evaluate_all_metrics(data_input, "K-means", label)
    comparison_results.append(metrics_km)

    # Run Hierarchical variant
    metrics_hc, labels_hc = evaluate_all_metrics(data_input, "Hierarchical", label)
    comparison_results.append(metrics_hc)
```

Figure 4.3 Run Experiment to Determine Best Model

To perform the comparison between various modelling strategies, the defined function is applied for the two algorithms. The metrics are appended in a list and transformed into a table for comparison, which is further described in the next subsection.

4.2 Comparison of evaluation metrics

This section will compare the evaluation metrics between different models. The dimension reduction results of PCA will be discussed, followed by UMAP.

--- COMPREHENSIVE COMPARISON MATRIX (PART 4) ---

	Method	Dim Reduction	Silhouette	Davies-Bouldin	Calinski-Harabasz	Optimal k	Runtime
0	K-means	None	0.3046	1.1160	1854.272600	6	0.0184
1	Hierarchical	None	0.5621	1.5873	1398.693000	2	5.4964
2	K-means	PCA (90%)	0.6773	1.3984	1815.569900	2	0.0117
3	Hierarchical	PCA (90%)	0.7833	0.6155	1607.019500	3	4.8116
4	K-means	PCA (95%)	0.3303	1.0442	2039.694200	6	0.0346
5	Hierarchical	PCA (95%)	0.7680	0.6468	1394.512800	2	4.3452
6	K-means	PCA (99%)	0.3092	1.1059	1881.475500	6	0.0144
7	Hierarchical	PCA (99%)	0.5100	1.6409	1362.121000	2	4.4701
8	K-means	UMAP	0.4833	0.8224	10325.871094	2	0.0053
9	Hierarchical	UMAP	0.4555	0.8776	9521.592773	2	3.1709

Figure 4.4 Comparison Matrix of Different Models

Figure 4.4 shows a comprehensive comparison matrix for different models. Looking at the results of K-Means clustering, the runtime is generally faster, which finishes the task in less than 0.02 second. Applying PCA 95% and 99% maintain the same optimal number of clusters (6) and improve the results slightly in terms of all 3 metrics, with 95% improving more than 99%. For instance, the Silhouette score has increased from 0.3046 (baseline) to 0.3303 (PCA 95%). Moreover, PCA 90% shifted the optimal k to 2, improving the Silhouette score to 0.6773, but making the other metrics (DBI and CHI) worse.

Hierarchical clustering with PCA 99% maintains the same number of optimal k but has lower performance metrics, suggesting that the PCA 99% does not help with noise reduction. On the other hand, PCA 95% significantly improved Silhouette score (from 0.5621 to 0.768) and DBI (from 1.5873 to 0.6468), which is likely to be the result of improvement in the cluster from both cohesion and separation. Lastly, PCA 90% shifted the number of optimal k to 3, and showed significant improvement in all three evaluation metrics as compared to baseline, and is the best PCA results under Hierarchical clustering. This means PCA has removed unnecessary noise for hierarchical clustering which modified how the clustering is done in baseline.

As for the dimension reduction with UMAP, the transition to a non-linear reduced feature space shifted the optimal k to 2, resulting in a higher Silhouette score. The DBI and CHI also show significant improvement compared to the baseline model. Notably, CHI increased to almost 5 times its previous score, indicating that the manifold projection has enhanced cluster separation and internal density of the cluster.

However, the result of the combination of Hierarchical Clustering with UMAP showed a complex set of performance metrics, as it has lowered the Silhouette score, but improved DBI and CHI. Similar to the case of K-Means + UMAP, the CHI increased significantly, which suggests that this metric is sensitive to UMAP dimension reduction, likely due to non-linear clumping effect of UMAP inflating the ratio of between-cluster dispersion to within-cluster dispersion. The improvement in DBI suggests that the UMAP created a more dense and compact neighbourhood, while the lowered Silhouette score suggests that the Hierarchical clustering algorithm fail to define clean boundaries between the non-linear shapes.

```
--- BEST CLUSTERING COMBINATIONS ---
Top Silhouette Score: Hierarchical + PCA (90%) (Score: 0.7833)
Top Davies-Bouldin Index: Hierarchical + PCA (90%) (Score: 0.6155)
Fastest Performance: K-means + UMAP (Time: 0.0049s)
```

Figure 4.5 Best Clustering Combinations

Figure 4.5 shows the best clustering combinations. In terms of evaluation metrics, Hierarchical clustering with PCA (90%) scores the best across all models in terms of Silhouette score (0.7833) and Davies-Bouldin Score (0.6155). The optimal k for this model is 3 and the runtime is 4.8s. On the other hand, the fastest model is K-Means with UMAP, which finishes the task in 0.0053 second.

4.3 Visualization and Analysis of clusters

The visualization and analysis of clusters is done for the best performing model, which is hierarchical clustering on PCA 90% at k=3. Table 4.1 details the distribution of final customer segments. Cluster 0 represents the majority of customers (8548) while cluster 1 (65), cluster 2 (23) contain lesser members. Visualization of the clusters on two main principal components is shown in Figure 4.6. From the diagram, it is clear that hierarchical clustering algorithms manage to cluster the different groups of customers with specific financial behaviour in the V shape structure as cluster 1 and 2. The remaining customers are grouped under the same cluster 0, which is a dense core of data points at origin (0,0), suggesting that these are likely the same customer with similar profile in Principal component 1 and 2.

Table 4.1 Distribution of Final Customer Segments

Cluster ID	Customer Count	Percentage (%)
0	8548	98.98
1	65	0.75
2	23	0.27

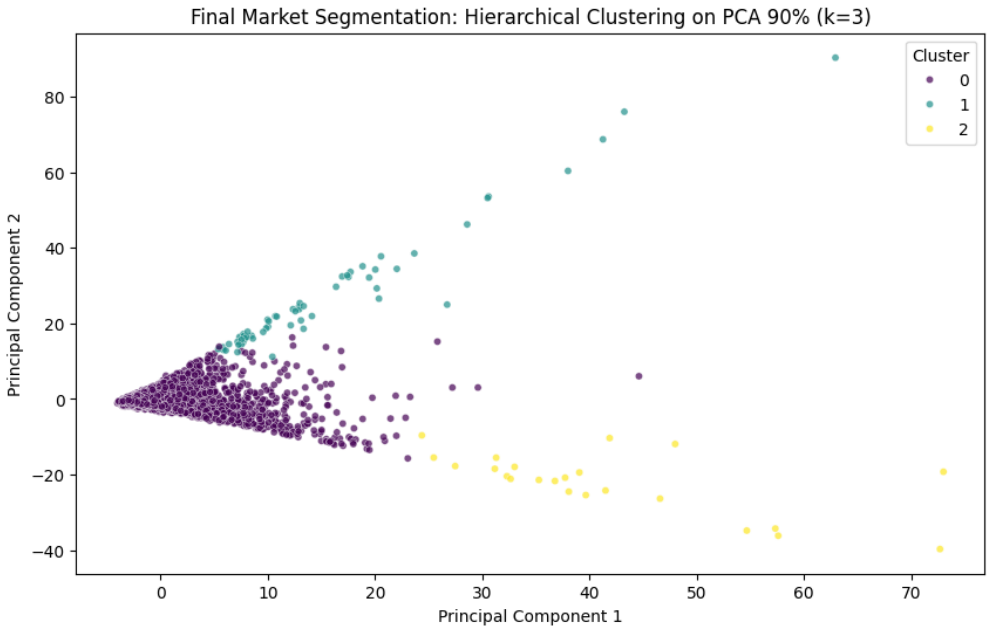

Figure 4.6 Final Market Segmentation: Hierarchical Clustering on PCA 90%

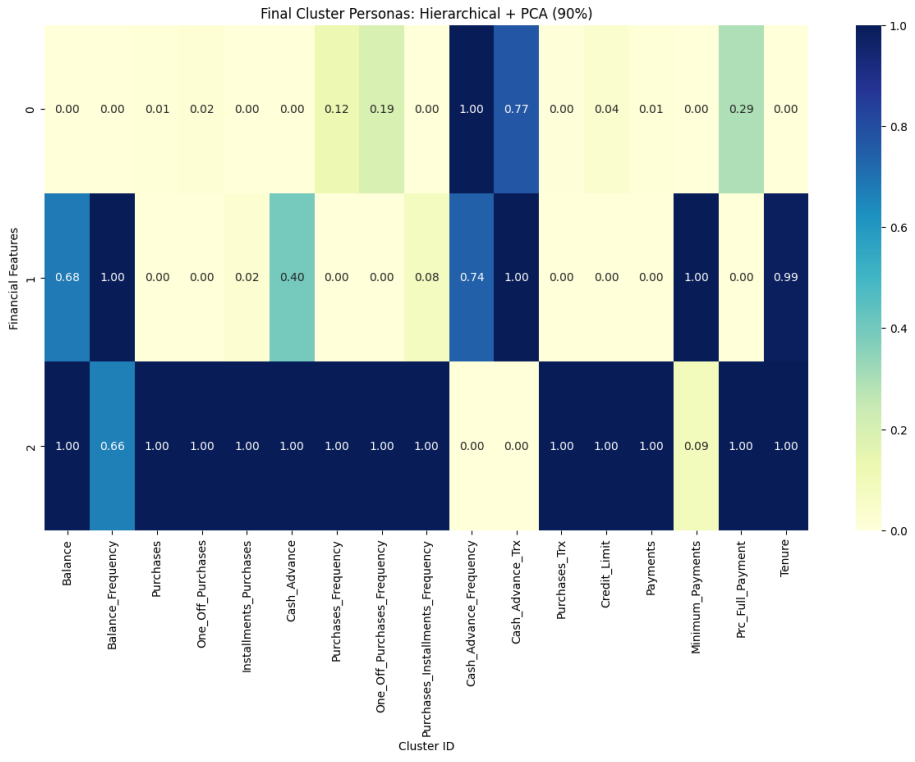

Figure 4.7 Final Cluster Persona: Hierarchical + PCA (90%)

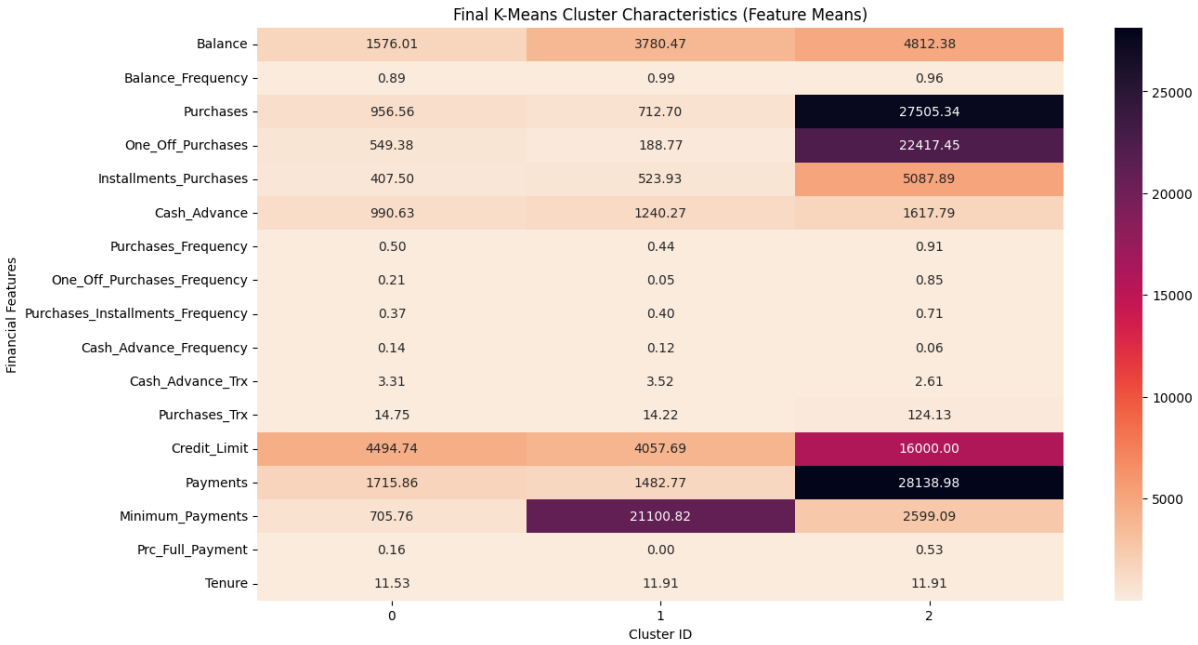

Figure 4.8 Final K-Means Cluster Characteristic (Feature Means)

Figure 4.7 and Figure 4.8 illustrate the characteristic of the clusters. Cluster 0 represents the majority of the credit card users, which have average values for all features with no specific financial behaviour observed. Cluster 1 is formed by at-risk customers with high minimum payments but low purchase. The current mean balance for this group is moderate ($712.7). The high minimum payments are reflecting their payments for outstanding balance rather than current month's spending behaviour. This information indicates this group consists of at-risk or defaulted users, where it is likely that their debt has been accumulating for a long time and these debts are recently cleared when the bank has requested settlement of accumulated past amounts. Cluster 2 are formed by high spender, with high mean purchase-related features ($27,505 for purchases, $22417 for one off purchase and $5087 for installment purchase). This group also has a high credit limit ($16,000) and payment ($28,138). This is the targeted group (VIP) for marketing strategy as they contribute a large amount to the bank revenue.

Section 5: Critical Analysis & Discussion

1. Did dimensionality reduction improve clustering quality? Why or why not?

Yes. From the perspective of evaluation metrics used in this project, dimension reduction techniques improve the results of the clustering algorithm. The baseline model (K-means and hierarchical) carries out the clustering with all 17 features (after removing customer ID and scaling), unfortunately the results of the baseline clustering were poor due to high dimensionality and feature redundancy. The K-means baseline has a Silhouette Score of 0.3046 and a DBI of 1.1160, and the hierarchical baseline has a Silhouette Score of 0.5621 and a DBI of 1.5873. The correlation heatmap reflects the high multicollinearity (PURCHASES and ONEOFF_PURCHASES with the 0.92 correlation). In the high-dimensional space (17 features/17 dimensions), the distance calculation for the unsupervised learning model becomes diluted because the models might give more weight to the redundant features (noise/almost the same attribute) and make the clusters overlap.

Scenario	Model	Silhouette	DBI	Optimal k	Result Quality
Baseline (None) (17 Features)	K-Means	0.3046	1.116	6	Confused
	Hierarchical	0.5621	1.5873	2	Confused
PCA (90%) (5 Components)	K-Means	0.6773	1.3984	2	Improved
	Hierarchical	**0.7833**	**0.6155**	**3**	**BEST (Enough Thinking)**
PCA (95%) (7 Components)	K-Means	0.3303	1.0442	6	Quality Drops
	Hierarchical	0.768	0.6468	2	Strong but declining
PCA (99%) (12 Components)	K-Means	0.3092	1.1059	6	Poor
	Hierarchical	0.51	1.6409	2	Over-thinking
UMAP (2D)	K-Means	0.4833	0.8224	2	High separation
	Hierarchical	0.4555	0.8776	2	High separation

Figure 5.1 Performance Metric table for All Possible Scenarios

Based on Figure 5.1, it shows that the high dimensions (17 features) will affect the distance, which will have less impact because each of the points is almost the same as the others. By reducing the dimensions to 5 (PCA 90%) or 2 (UMAP), the model can calculate a clearer boundary between the clusters with the support of performance metrics like lower Silhouette and higher DBI. PCA merged those redundant/repeated features into a single component, and it allowed the hierarchical model to reach the highest Silhouette of 0.7833 because the model was not confused by the repetitive data. For the PCA, the 90% and 99% difference lies in the model's feature processing. At 90%, the model has a clearer view for the data points in the feature space and focuses on the most significant dimension, denoising the data to reveal core customer behaviour (e.g. purchases related metrics reduced to one principal component). At 99% the model overfit to the variance obtained which contains noises. PCA 90% with the 5 components can represent 90% of the information, where just 10% of the data is filtered out

as random noise. As the DBI for the hierarchical clustering drops to 0.6155, it means that the cluster is compact and far away from the clusters. PCA 99% with the 12 components that reintroduced the noise into the model that negatively impact the clustering process in terms of Silhouette Score drop from 0.7833 to 0.51, which means that the more data that is kept will make the model overfit the insignificant details and lead to overlapping issues.

Figure 5.1 indicates that generally by reducing the dimension with UMAP, the Euclidean distance used by K-Means and the Ward linkage used by hierarchical clustering become more accurate, and it reflects the CHI that reaches more than 10000 for UMAP, which means the dimension reduction created the denser clusters. There is a slight drop for Silhouette score for hierarchical clustering, likely due to the limitation of the same UMAP best parameters of K-Means being used, which may be improved with further tuning.

In summary, the dimensionality reduction techniques improve the performance of the model from confused “thinking” (baseline) due to noise to enough “thinking” (PCA 90%), where “thinking” is used as an analogy to explain the clustering process for easier understanding. By filters 10% of the redundancy of the dataset, and it achieves the highest Silhouette Score of 0.7833 and lower DBI score of 0.6155.

2. Which combination of clustering + dimensionality reduction worked best?

Based on Figure 5.1, hierarchical clustering with PCA (90%) scores the best in terms of Silhouette score (0.7833) and Davies-Bouldin Score (0.6155). The agglomerative clustering is able to define average and niche customer groups accurately on the summarised data (PCA). It is also supported by the 2D and 3D illustration of the PCA, where the visible clusters by human interpretation are correctly identified by the model. The UMAP provides great visualisation, but the PCA 90% provides the most stable mathematical separation for the hierarchical model.

3. What meaningful patterns emerged from your clusters?

Identified the distinct and actionable financial behaviours and depends on the specific group behaviours, which discover the personality of the credit card user as an average user, high spender or risky customer/debtor. By applying the dimension reduction, eliminate the distraction of the 17 different features and identify the financial personality of the customer. The average user is the majority (mainstream), which means the people with the basic credit card application. At-risk (debtors) are the people that are not buying any new items but paying off massive old bills. The VIPs (the profit users) are the people that spend huge amounts on luxury with the high limit. Cluster 1 is the hidden insight where, without the clustering, we are unable to identify the users that are clearing old debt that was requested by the bank, as the minimum payment is about 21k, which means they are defaulted users with high risk. Cluster 2 is the business target, as this cluster is the revenue engine, with purchases

of about 27k, which is 3000% higher than the average. If we are using the unsuitable model that will overlap the cluster, it will affect the business of the bank, and the bank might treat the VIP the same as the debtor, and it will lead to failure in the decision-making.

4. Can you interpret and label the clusters based on feature characteristics?

Imagine that the mainstream is the background of the bank, At risk are the red flags and the VIPs are the gold stars.

Cluster 0 is the standard transactor (baseline), which is the majority of the users with an average balance and moderate spending. The customer in cluster 0 is using the card for the daily/routine expenses and will not fully exceed the credit limit or carry the extreme debt.

Cluster 1 is the debt revolvers (high risk with the red flags), which have a moderate balance ($712.7), high minimum payment and low current payment. These are at-risk customers, where they are focused on clearing the old debt with minimum payment rather than making the full payment for the balance. The red flags for cluster 1 are because they are paying the interest to the bank with the high default risk.

Cluster 2 is the high net worth VIP (top tier), which has the massive purchases ($27505), high credit limit ($16000) and high total payment ($28138). The customer in cluster 2 drives the transaction volume and merchant fee revenue. The high payment indicates that they are rich enough to clear all the balance despite making high purchases.

5. Provide business/domain insights based on your findings

Risk mitigation of cluster 1 allows the bank to avoid offering the new credit limit increases to customers in cluster 1, as the higher credit limit and the higher debt lead to an inability to clear the debt in the time given. The bank should offer the balance transfer event or debt consolidation loan (refinancing strategy) with the fixed interest rate to ensure the balance is recovered without the total default.

For cluster 2, it is the revenue optimization, as the customers in cluster 2 spend around $27k annually and have a high credit limit ($16k). The bank should offer the exclusive concierge service, travel insurance or higher cashback rewards for retention and upselling. Invite the customer in cluster 2 for premium/private banking tier and offer the cashback reward for the high one-off purchases to maintain the loyalty of the customer.

For the cluster 0 (mainstream), the majority (98% of the users) have the average, as they are stable without the high profit. The growth and engagement strategy can be applied for these cases. By using the marking to incentivize the small instalment purchases to gradually increase the credit cards utilization without pushing them into debt, such as promoting the 0%

interest for short term 6-month instalment offers for the electronics or furniture, which are a safe action to promote growth and enhance customer loyalty.

6. Discuss computational efficiency vs clustering quality.

Depends on the priority of the task. K-means is like a calculator that is fast and able to get the job done instantly with some simple assumptions. Hierarchical and UMAP are like the supercomputer that takes longer time to think/process, but it is able to produce more detailed and accurate results. By using PCA methods. The model is able to run faster.

Method Combination	Dim. Reduction	Silhouette (Quality)	Runtime (Speed)	Result Category
K-Means + UMAP	UMAP	0.4833	0.0049s	Fastest Performance
K-Means + PCA (90%)	PCA	0.6773	0.0102s	High Speed / Good Quality
Hierarchical + PCA (90%)	PCA	0.7833	4.6162s	Best Quality / Slowest
Baseline Hierarchical	None	0.5621	4.2367s	Poor Quality / Slow

Figure 5.2 Comparison Table of the Speed and Quality.

Based on Figure 5.2, Hierarchical is the quality over speed (slower) because the hierarchical agglomerative clustering has the computational complexity. It took more than 4 sec to process 8950 records, but if the bank had 1 million or more customers, then the model would take more than 1 day to run. If the situation allows us to have enough time to wait for the results, then hierarchical clustering is good because of the highest Silhouette Score (0.7833) and accuracy. It can be used to determine the VIP spenders for the high-stakes business decision.

Based on Figure 5.2, K-means has linear complexity, which means speed over quality. With the application of the PCA (90%), it took 0.01 s, while the Silhouette Score (0.6773) is lower than the hierarchical (0.7833). The K-means is better for the real-time system where the system needs to categorise/identify the customer in the second swipe of the card.

7. For your specific dataset, which approach would you deploy in production?

K-Means + PCA (90%) for the real-time system scalability. Hierarchical + PCA (90%) for the high-accuracy strategy. K-Means + PCA (90%) is able to operate fast and efficiently,

especially when the bank needs to identify the second the customer swipes the card (real-time processing). Hierarchical + PCA is slower but more precise and accurate than what is used in the massive marketing strategy planning for next year (batch-processing).

8. What are the limitations of your analysis?

The first limitation is that the business objectives are not given in this case. In real world practice, the optimal number of clusters will change based on the end goal. For example, the optimal k differs when the objective is to determine high risk credit card customers, high spenders or for general marketing purposes.

Another significant limitation of this study is the optimal number of clusters (k) is chosen based on a single evaluation metric, Silhouette score. While this automation approach reduces the need for human interpretation (visualization of clustering quality) and simplifies the process of selecting the best model based on different evaluation metrics (silhouette vs DBI vs CHI), it may overlook the hidden insight of the dataset. In this project, silhouette score often favours a lower k because a greater separation between clusters results in higher silhouette score, where using the other evaluation metrics may have a different conclusion. However, some business cases might require more granularity for targeted marketing, which may require human interpretation to choose the best model.

The analysis is limited by the data bias and structural data issues. Data bias is the high correlation between the purchase feature (0.92) that forces the model to put more weight on the spending behaviour than on other financial indicators. Structural data issues include the small sample provided as cluster 1 (At-Risk) with only 13 customers. It leads to the imbalance of the class where there are 8950 records in the dataset but only 13 people with the red flags of fragility. If 1 or 2 typos or wrong keyings for the 13 people will make the significant issues which make the result sensitive to outliers.

Lastly, due to the longer computational time for GridSearch with UMAP, the best UMAP parameters are selected based on K-Means clustering results and applied for both algorithms, with the assumption that the same set of parameters work best for both clustering algorithms. In best practice, another set of best UMAP parameters should be selected for hierarchical agglomerative clustering.

Reference

Habibimoghaddam., F. (2023). *Customer credit card data* [Data set]. Kaggle. https://www.kaggle.com/datasets/fhabibimoghaddam/customer-credit-card-data

Scikit-learn developers. (2025). *sklearn.metrics.silhouette_score*. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html